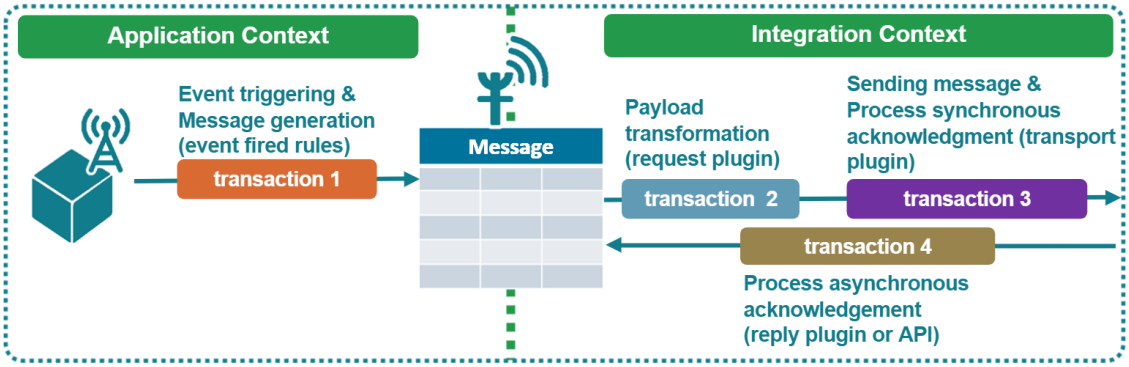
Acknowledging Messages

Guidewire servers and messaging transactions



- **Message acknowledgement** is the process in which Guidewire interprets a messaging response from an external system
- All activity from **acknowledging messages** takes place in either **transaction 3 or 4**
- Messaging occurs across multiple transactions
  - If an exception is thrown, work in the current transaction is rolled back

Notes

Guidewire servers and messaging transactions

The servers involved at each point of the process are important because they determine where payload work should occur. The majority of payload generation should be done in Event Fired rules on the **ui** role server that manages the given user's session. However, if some portion of payload generation must be done immediately before the message is sent, that portion of the payload can be generated on the **messaging** role server.

A message is always processed in at least two and as many as four transactions:

- Transaction 1 – Creates the message and generates the payload. This transaction always occurs.
- Transaction 2 – Prior to sending the message, an optional transaction may transform the message payload. This is usually done when the message must include information that can only be known immediately before sending the message.
- Transaction 3 – The sending of the message to the external system occurs in its own transaction. This transaction always occurs. If the external system acknowledges the message synchronously, then the acknowledgement is included in this transaction.
- Transaction 4 – If the external system acknowledges the message asynchronously, then a separate transaction processes the acknowledgement. This transaction does not occur for messages that are acknowledged synchronously.

The transaction structure is important because this determines the work that is rolled back if an error occurs.

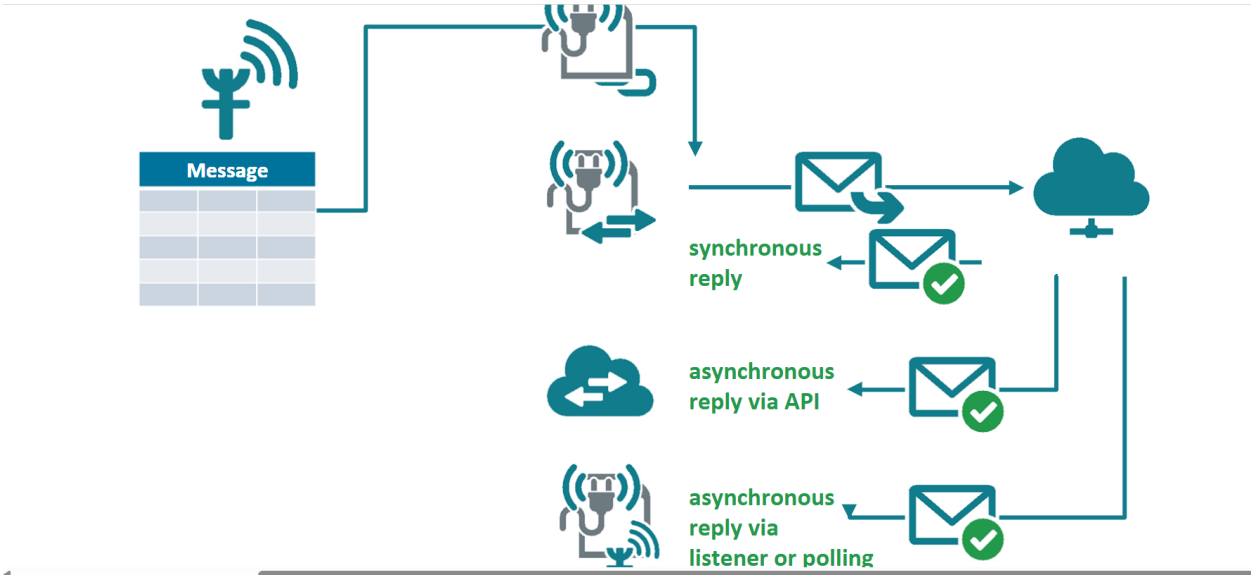
Guidewire expects that every sent message will immediately or eventually be followed by a response from the external system. An immediate response is a synchronous response. An eventual response is an asynchronous response. There, is of course, the possibility that the external system never replies with a response. In some cases, a response is simply a reply that is a positive acknowledgement. In other cases, a response includes both an acknowledgement and some type of data such as a report.

An acknowledgement comes in the form of a positive acknowledgement or negative acknowledgement. With regards to the messaging Guidewire API, integration code calls an acknowledgement or reports an error. Code that calls an acknowledgement is a positive acknowledgement. Code that invokes an error is generally a negative acknowledgement.

The reply mechanism for the external system to make a response often depends if the response synchronous or asynchronous. For a synchronous response, often the message transport plugin is the reply mechanism. For asynchronous responses, often a Guidewire Application published web service or a message reply plugin is the reply mechanism.

Message acknowledgment

Plugins that acknowledge messages



Notes

Plugins that acknowledge messages

Every destination has up to three plugins that are used for messaging. Two of these plugins could play a role in acknowledging a message, though it is possible that no plugin is used for acknowledgement.

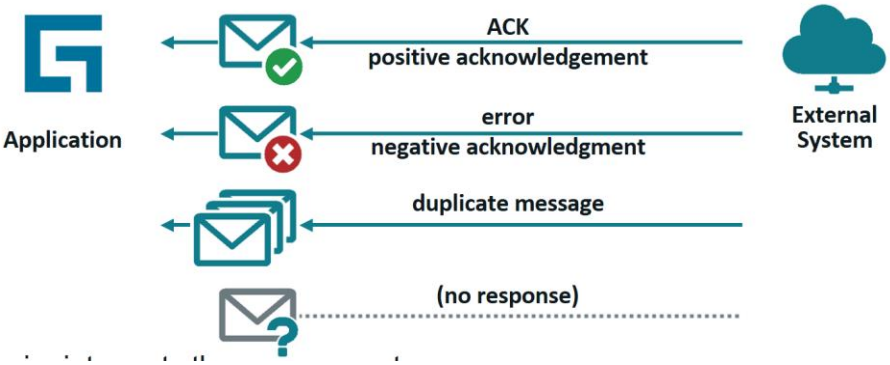
The request plugin is an optional plugin that is responsible for payload transformation. It is not involved in the acknowledgement of messages.

The transport plugin is a required plugin that is responsible for sending the message to the external system. If the acknowledgement is received synchronously, then this plugin also processes the reply.

An external system can acknowledge a message using a remote call (either the IMessageToolsAPI or a custom-built web service). If a remote call is used, then the message reply is handled independent of the destination's plugins.

The reply plugin is an optional plugin that initializes a listener queue and associates it to the destination. It processes acknowledgements sent asynchronously using the listener queue. Like predefined plugins, messaging plugins can be implemented in Gosu or Java.

Interpreting the external system response



- Guidewire interprets three response types
  - Positive acknowledgement, error, and duplicate response
- No response may also require handling

Interpreting the external system response

Guidewire can interpret a response to be one of three types: positive acknowledgement, error, and duplicate response. Guidewire applications must also decide how to react when the external system never sends a response to a message.

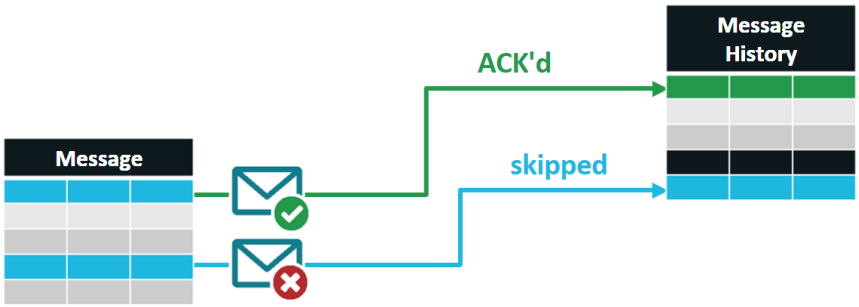
Message and MessageHistory

Message table contains:

- Unsent messages that…
  - Have yet to be sent
  - Failed send
- Sent messages that are…
  - Pending acknowledgement
  - Waiting to be retried
- Errored messages that…
  - Require manual intervention

MessageHistory table contains:

- Positively acknowledged messages
- Skipped messages
- Retried messages



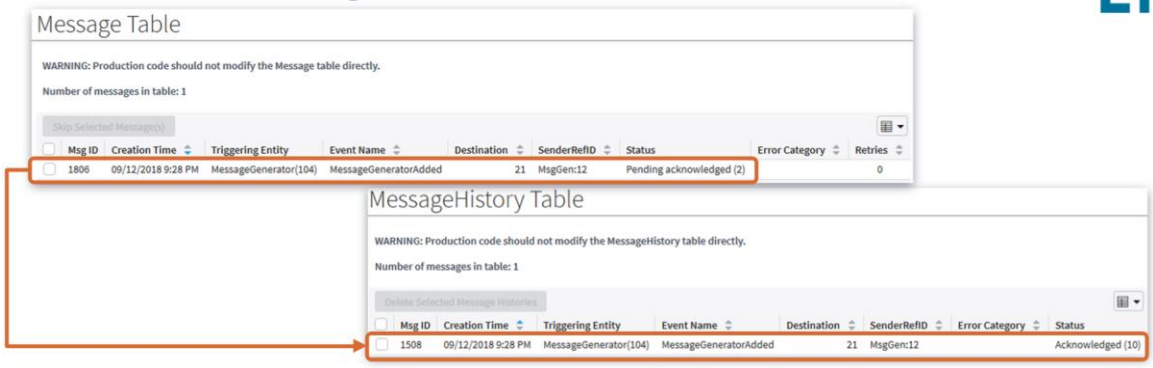
Notes

Message and MessageHistory

A message can move from the message table to the message history table if the message is positively acknowledged or skipped. The Message table contains messages that need further processing. The MessageHistory table contains messages that do not need further processing. A message can move from the message table to the message history table if the message is positively acknowledged or skipped. Otherwise, the message remains in the message table.

Over time, the message history table accumulates a large number of message histories and therefore needs to be pruned on a regular basis. The MessageToolsAPI has a purgeCompletedMessages method that can be used to do this. For more information on pruning the MessageHistory table, refer to the System Administration Guide.

Positive acknowledgements



- A **positive acknowledgment (ACK)** is when the external system reports success
  - Message was successfully received and processed
- Guidewire application response to ACK
  - Moves the message from Message to MessageHistory
  - Changes the status to reflect acknowledgement

Positive acknowledgements

When the external system reports a message was successfully received and processed, Guidewire interprets this as a positive acknowledgement, also known as an ACK. When a message is positively acknowledged, Guidewire:

- Moves the message from Message to MessageHistory, and
- Changes the status to reflect acknowledgement

Status is an integer field on both Message and MessageHistory. In most cases, developers do not need to know what each integer represents. However, the statuses are defined in the Data Dictionary. The **Status** column in the Message Table and MessageHistory Table screens includes both the integer code and its meaning.

Message table status descriptions:

1 = Pending send. Waiting to send message.

2 = Pending ack. Message sent. Waiting for acknowledgment.

3 = Error. Deprecated.

4 = Retryable error. Error reported. Can be send exception or negative acknowledgment from external system.

Message History table descriptions:

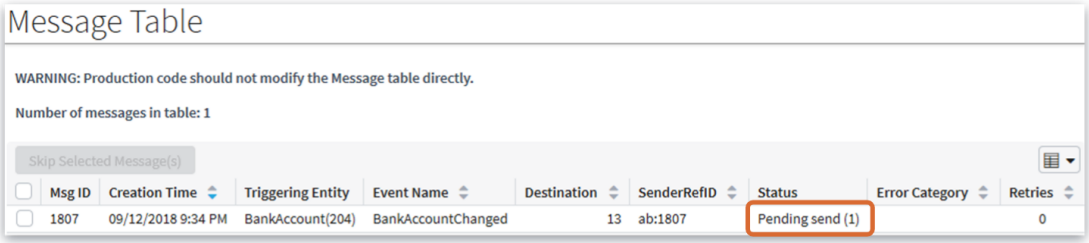
10 = Acked. Positive acknowledgment of message.

11 = Error cleared. The message was in retryable error state, but the administrator skipped the message.

12 = Error retried. Message with error retried. Clone of the original message created in the message table and will be resent.

13 = Skipped. Message was not in error – it was either Pending send(1) or Pending ack(2).

Messaging exceptions during send



- Initial send from Guidewire transport throws an exception
- Guidewire application response
  - Message stays in Message table
  - Message status remains as Pending send (1)
  - Guidewire retries to send the message until reaching the maximum number of retries as defined in the destination
  - At maximum retries, the Guidewire suspends the destination

Notes

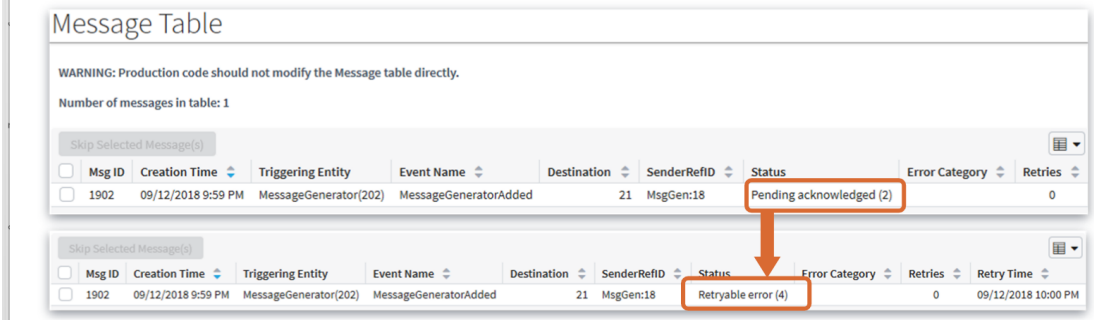
Messaging exceptions during send

An exception is different than a reported error for a message. There are various types of exceptions such as a java.net.ConnectionException or gw.xml.ws.WsiAuthenticationException.

When an exception is thrown in a transport plugin send method, the Guidewire application response is to retry sending the message until reaching the maximum number of retries. The message's destination configuration defines the value for the maximum number of retries.

In order for an additional messages to be processed for the messaging destination, you need to skip any messages reporting an error and to resume the suspended destination. An administrator can manually skip a message. It is also possible for custom code to skip messages, for example, from within a batch process. If a message is safe ordered, the next message for the destination/entity pair will not be sent until the message is either resent and successfully acknowledged or skipped.

Messaging errors after send



- Error from external system is known as a Negative Acknowledgment (NACK)
  - Temporarily cannot process the successfully received message
  - Reports error
- Guidewire application response
  - Message stays in Message table
  - Message status of retryable error
- Retry scenario
  - Manual retry from administrator or integration code to resend message

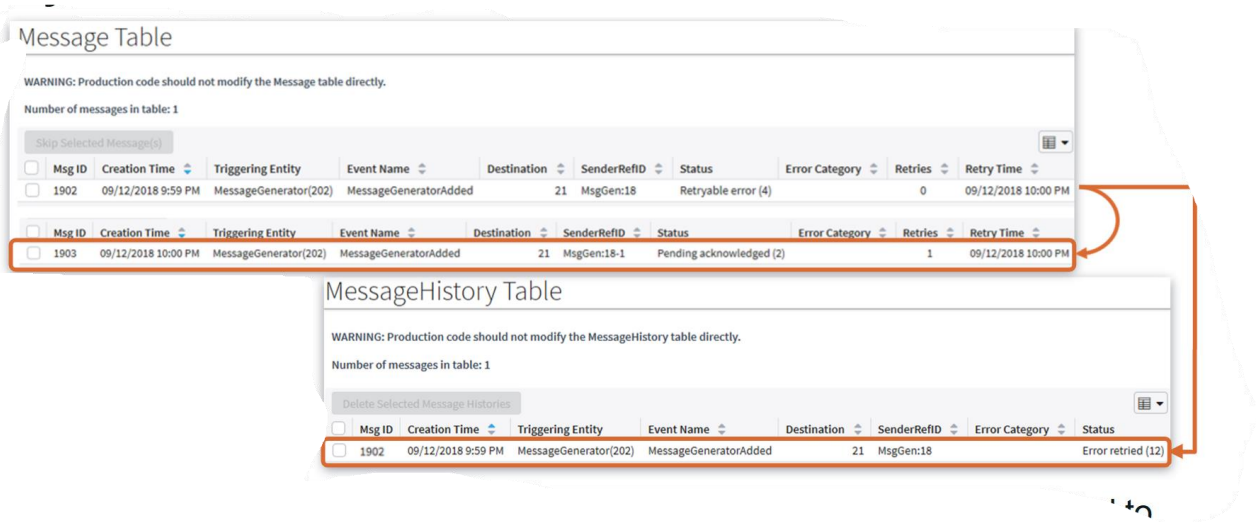
Notes

Messaging errors after send

Guidewire applications consider all messages with an error as retryable messages. A status of a retryable error indicates a condition that is expected to be temporary for the messaging destination. Some examples of temporary errors are: the external system is unreachable due to network problems, authentication protocols require new credentials, database locks, and the file system is full. In all cases, Guidewire application considers the messaging error as a retryable error.

The term **negative acknowledgement** (NACK) indicates that the external system is reporting an error. The mechanism for a negative acknowledgement is similar to an acknowledgement with respect to if the messaging architecture is synchronous or asynchronous. If the messaging is synchronous, the negative acknowledgement occurs in the transport plugin. If the messaging is asynchronous, a reply plugin, or a remote API reports the error.

Retrying an error



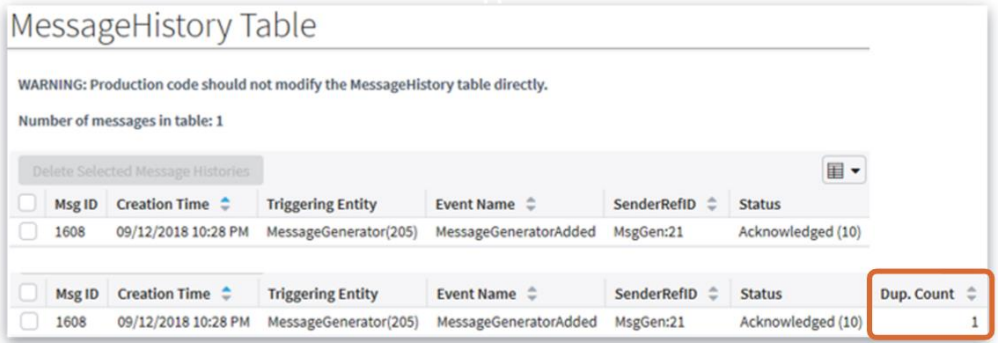
- A copy of the original message is created and sent
  - References the same event and triggering entity
  - Uses a different SenderRefID
  - Incremented RetryCount field by 1
- Original message is moved to MessageHistory table

Notes

Retrying an error

Note that in the screenshot of the Message table, there are two messages, one with an ID of 1902 and one with an ID of 1903. The first message had a status of **Pending acknowledged** and then had a status of **Retryable error**. When the error was retried, message 1902 was moved to the MessageHistory table, and message 1903 was created and sent. Even though Message 1903 has a different SenderRefID, it is linked to the same triggering entity **MessageGenerator(20)**. So, message 1902 and message 1903 are two separate message instances that relate to the same logical business event.

Duplicate messages



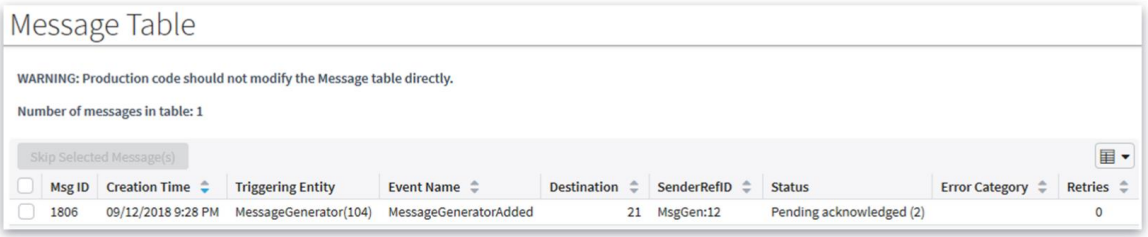
- Duplicate messages
  - One message sent successfully
  - Received by the external system
  - External system processes the same message more than once
  - External systems responds with a positive acknowledgement (ACK) per each processed duplicate
- Guidewire application response
  - Increments the Duplicate Count (Dup. Count) in the Message History table

Notes

Duplicate messages

Guidewire does not treat a message with a non-zero duplicate count any differently from a message with a zero-duplicate count. However, the recommendation is to process duplicates reported by the external system so as to retain as much information as possible about what was sent from the external system.

No response



- Guidewire application sends message to external system
  - No ACK or NACK from external system
- Guidewire application response
  - Keeps the message in the Message table
- Consider batch process to purge message table and avoid manual intervention
  - Apply logic for overdue messages

Notes

No response

The Message table should be checked for messages that have not received a reply within a timely fashion. This could be a Guidewire custom batch process or a database tool that monitors the Message table. Each non-response message typically requires manual evaluation to determine the cause of the non-response and the appropriate steps to fix the situation.

Contrasting retry scenarios

	Initial send error (Automatic retry during send)	External system error (Automatic retry after send)	Manual retry
What is the problem?	Exception thrown in **beforeSend** or **send** methods	External System reports an error	Message needs manual intervention
What executes the retry?	Internal Guidewire behavior	Integration code	Administrator using Retry button
Should each attempt use the same SenderRefID	Yes	No	Yes
What specifies the max retry number?	Max Retries property for messaging destination	Integration code using plugin parameter or script parameter	n/a
What happens when the max number of retries is	The destination is suspended	Integration code reports error with error category	n/a

Notes

Contrasting retry scenarios

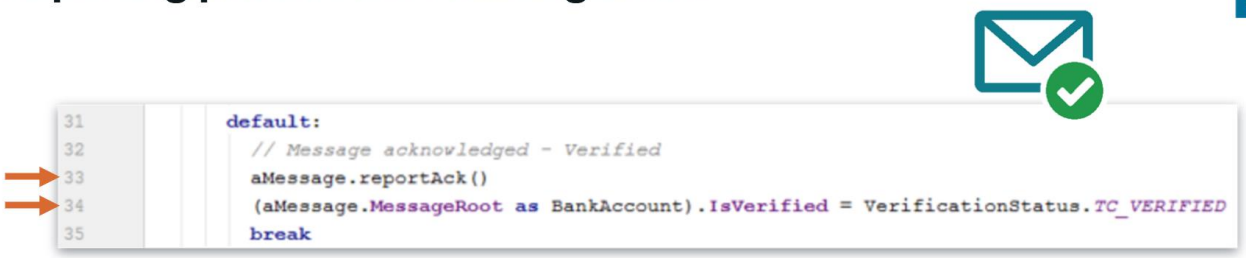
A message could be retried automatically **during the sending** of the message. A message could be **retried manually**. Both are discussed in detail in the Sending Messages lesson.

A message can be retried automatically **after the sending** of the message.

- Occurs when the external system reports a retryable error. For example, the message was received but not successfully consumed. The error is expected to be temporary, such as unavailable network resource.
- The message should be retried automatically by the integration code. When the code reports an error, it should specify a retry time.
- Even though it pertains to the same message event, the retry is a different message and the same SenderRefID should **not** be used. If the same SenderRefID is used, then the external system may consider the second attempt to be a duplicate message and disregard it.
- The maximum number of retries is specified by the integration code as defined by plugin parameter (for the transport or reply plugin) or a script parameter.
- If a given message exceeds the maximum number of retries, then the message should be reported with an appropriate error category, an administrator should be alerted, and no further automatic attempts should be made. Any other messages for the same destination should not be affected.

Configuring message acknowledgment

Reporting positive acknowledgement



```
31          default:
32            // Message acknowledged - Verified
33            aMessage.reportAck()
34            (aMessage.MessageRoot as BankAccount).IsVerified = VerificationStatus.TC_VERIFIED
35            break
```

- In some cases, Guidewire applications use messaging to request data such as reports from external systems, such as:
  - A police report detailing an accident or burglary of insured items
  - A medical report detailing customary prices for given medical procedures
  - A fraud report detailing that a claim for a given loss has already been filed
- In these cases, the external system responds with the report data and acknowledges the message

Notes

Reporting positive acknowledgement

Code example:

Line 33 - Message is acknowledged using reportAck method.

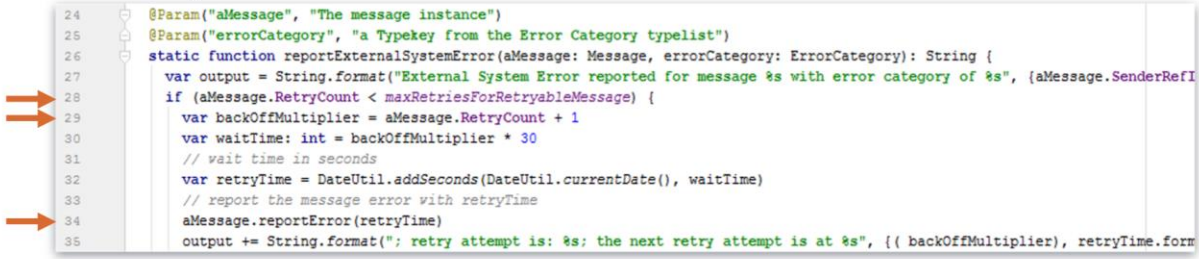
Line 34 - Bank account status updated to Verified.

Reporting errors after send below max retries (Examples)

- Code example 1

```
13  static property get maxRetriesForRetryableMessage(): int {
14    var value: int = 3
15    if (ScriptParameters.MaxRetriesForRetryableMessage > 0) {
16      value = ScriptParameters.MaxRetriesForRetryableMessage
17    }
18    return value
19  }
```

- Code example 2



```
24    @Param("aMessage", "The message instance")
25    @Param("errorCategory", "a Typekey from the Error Category typelist")
26    static function reportExternalSystemError(aMessage: Message, errorCategory: ErrorCategory): String {
27      var output = String.format("External System Error reported for message %s with error category of %s", {aMessage.SenderRefI
28      if (aMessage.RetryCount < maxRetriesForRetryableMessage) {
29        var backOffMultiplier = aMessage.RetryCount + 1
30        var waitTime: int = backOffMultiplier * 30
31        // wait time in seconds
32        var retryTime = DateUtil.addSeconds(DateUtil.currentDate(), waitTime)
33        // report the message error with retryTime
34        aMessage.reportError(retryTime)
35        output += String.format("; retry attempt is: %s; the next retry attempt is at %s", {( backOffMultiplier), retryTime.form
```

Notes

Reporting errors after send below max retries (Examples)

Code example 1

Line 16
- Custom parameters needed for re-send retry behavior.
  - Defined for maximum retries, retry delay etc.
  - Plugin parameters (for reply code in plugins).
  - Script parameters (for reply code in web services).

Code example 2

Line 28
- Checks if below max retries.

Line 29
- RetryCount property starts at 0 and increments with each resend.

Line 34
- Call reportError(retryTime) before maximum retries limit reached.
- Application sends a cloned message at the future retry time.

The code references a method that you can find in the trainingapp.demo.messageack .MessageAckUtil class in TrainingApp. The reportExternalSystemError method illustrates how an external system can report an error using a remote API call or message reply plugin.

A backoff multiplier is a value that is used to increase a retry interval arithmetically. In the example above, the backoff multiplier simply increases the wait time by 60 seconds per iteration. For the first resend, aMessage.RetryCount is 0, so backOffMultiplier is set to 1 and waitTime is set to 30 seconds. For the second resend, aMessage.RetryCount is 1, so backOffMultiplier is set to 2 and waitTime is set to 60 seconds. For the third resend, waitTime is set to 120 seconds, and so on.

Every destination should have a specific number of times a message can be retried after the initial send. The number of retries should be specified using either a plugin parameter for the reply code in the transport or reply plugin or a script parameter for the reply code in the web services handler.

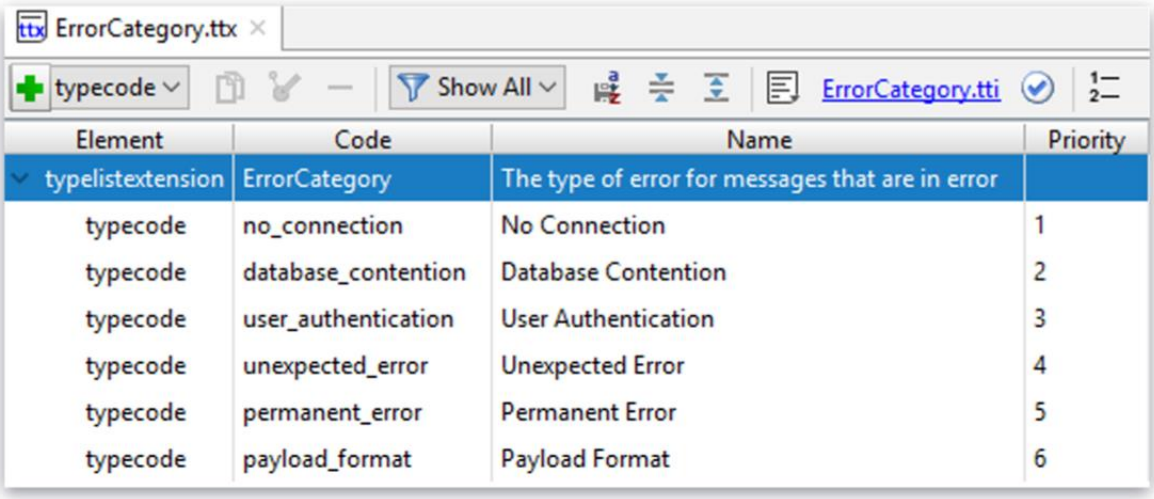
Before the max has been reached, report the error using message.reportError(whenToResendMessage). Then, when sending the retry message, use a new SenderRefID even though it pertains to the same message event, it must be treated as a new message. The amount of time between resends should increase with each resend.

If a message is resent two or more times because the external system reported a retryable error, each message should use a different SenderRefID. This is because the external system may treat two messages with the same ID as the same message. In other words, the external system may be "duplicate aware". If the second message uses the same SenderRefID as the first, then the external system may ignore it or report it as a duplicate, which would prevent the message from ever getting processed.

You can also manually retry any message using the Event Messages administration screens. This is discussed later in this lesson.

Reporting errors after send at max retries

- Define typekeys in ErrorCategory typelist extension
  - …config.extensions.typelist.ErrorCategory.ttx
- `message.reportError(errorCategory)`
  - Call when retry maximum retries limit reached
  - Specify the error category
  - Application no longer tries to resend the message
  - Alert an administrator



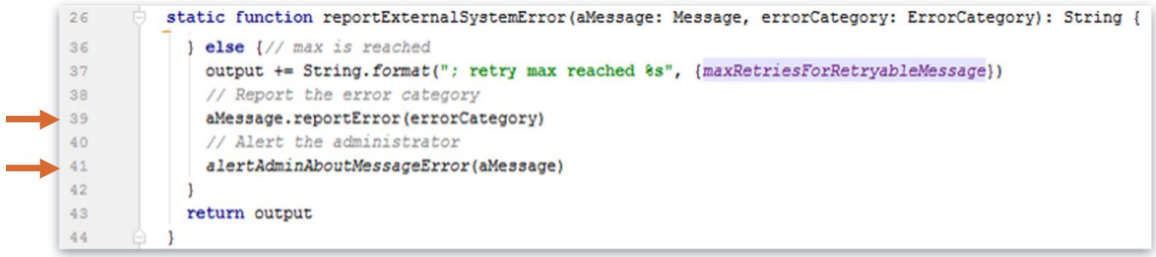
Element	Code	Name	Priority
typelistextension	ErrorCategory	The type of error for messages that are in error	
typecode	no_connection	No Connection	1
typecode	database_contention	Database Contention	2
typecode	user_authentication	User Authentication	3
typecode	unexpected_error	Unexpected Error	4
typecode	permanent_error	Permanent Error	5
typecode	payload_format	Payload Format	6

Notes

Reporting errors after send at max retries

ErrorCategory is a typelist that identifies the error reason for a message. In each base application, the typelist is empty. When a given message's retry max has been reached, report the error using message.reportError(errorCategory) and alert an administrator to the issue as it pertains to the message error. Guidewire will no longer try to resend the message. When you simply report the error, but no exception is thrown, Guidewire will not suspend the message destination.

Reporting errors after send at max retries (example)



```
26  static function reportExternalSystemError(aMessage: Message, errorCategory: ErrorCategory): String {
36    } else {// max is reached
37      output += String.format(": retry max reached %s", {maxRetriesForRetryableMessage})
38      // Report the error category
39      aMessage.reportError(errorCategory)
40      // Alert the administrator
41      alertAdminAboutMessageError(aMessage)
42    }
43    return output
44  }
```

The code references a method found in the **trainingapp.demo.messageack.MessageAckUtil** class in TrainingApp

Notes

Reporting errors after send at max retries (example)

Code example

Line 39

- Call message.reportError(errorCategory) when the maximum retries limit reached.
- Application no longer tries to resend the message.

Line 41

- An administrator alert usually takes the form of:
  - An email sent to the administrator, and/or
  - An activity assigned to the administrator

A backoff multiplier is a value that is used to increase a retry interval arithmetically. In the example above, the backoff multiplier simply increases the wait time by 60 seconds per iteration. For the first resend, aMessage.RetryCount is 0, so backOffMultiplier is set to 1 and waitTime is set to 60 seconds. For the second resend, aMessage.RetryCount is 1, so backOffMultiplier is set to 2 and waitTime is set to 120 seconds. For the third resend, waitTime is set to 180 seconds, and so on.

Reporting duplicates



```
66  @Param("senderRefID", "The SenderRefID of the message")
67  static function reportDuplicate(senderRefID: String): String {
68    var output: String
69    var aMessageHistory = Query.make(MessageHistory).compare(MessageHistory#SenderRefID, Relop.Equals, senderRefID).select().AtMostOneRow
70    if (aMessageHistory != null) {// history found so duplicate
71      Transaction.Current.add(aMessageHistory)
72      aMessageHistory.reportDuplicate()
73      output = String.format("Duplicate reported for MessageHistory with SenderRefID: %s with count %s", {aMessageHistory.SenderRefID, aMessageHistory.D
74    } else {
75      // production code would throw an exception here
76      output = String.format("There is no MessageHistory in the MessageHistory table for the message with SenderRefID", {aMessageHistory.SenderRefID})
77    }
78    return output
79  }
```

- Duplicates can be reported only from MessageHistory instances
- When an external system reports a duplicate, the integration code should find the related entry in the MessageHistory table and report the duplicate
  - The only effect this has is incrementing the MessageHistory entry's **DuplicateCount** integer

Notes

Reporting duplicates

Code example

Line 69
- Query MessageHistory table to report duplicates from existing MessageHistory instance.

Line 72
- Call messageHistory.reportDuplicate method
  - One message sent successfully and received by the external system.
  - External system processes the same message more than once.
  - External systems responds with a positive acknowledgement (ACK) per each processed duplicate.

Best practice
Guidewire does not treat a message with a non-zero duplicate count any differently than a message with a zero-duplicate count. However, the recommendation is to process duplicates reported by the external system so as to retain as much information as possible about what was sent from the external system.

The code references a method that you can find in the trainingapp.demo.messageack.MessageAckUtil class in TrainingApp.

As shown later in this presentation, the MessageReply plugin interface provides a message finder utility to eliminate the use of an explicit Query against the Message and MessageHistory tables.

Reporting messages without responses

- Check the Message table with:
  - Batch process, or
  - Database monitoring tool
- Evaluate the cause and handle the resolution
  - Alert via email, activity, flag entry etc.

FlagOverdueLegalReportsBatch.gs

```
8   class FlagOverdueLegalReportsBatch extends BatchProcessBase {
9     construct() { super(BatchProcessType.TC_FLAGOVERDUELEGALREPORTS) }
12
13    override function doWork() {
14      // Any batch process that modified data must manually create
15      // a bundle.
16      Transaction.runWithNewBundle(\bundle -> {
17        print("Starting Flag Overdue Legal Reports batch process")
18        var numberOfContactsWithOverdueReports = 0
19        // Query for legal report messages that have been waiting too
20        // long for an acknowledgement. In order to produce interesting
21        // results in a classroom setting, the threshold time is set to
22        // 15 seconds before the current time.
23        var thresholdTime = DateUtil.addSeconds(DateUtil.currentDate(), -15)
24        var messageQuery = Query.make(Message)
25        messageQuery.compare("SendTime", LessThanOrEquals, thresholdTime)
26        messageQuery.compare("DestinationID", Equals, 15)
27        var resultsObj = messageQuery.select()
28        for (currentMessage in resultsObj) {
29          var relatedContact = (currentMessage.MessageRoot as ABContact)
30          // Do not flag a contact if it is already flagged for an overdue legal report.
31          if (!relatedContact.FlagEntries.hasMatch(\currentEntry ->
32              currentEntry.Reason == FlagEntryReason.TC_OVERDUE_LEGAL_REPORT)) {
33            // relatedContact was queried for, and therefore is in a read-only bundle.
34            // Copy the object into the current, writeable bundle.
35            relatedContact = bundle.add(relatedContact)
36            var newEntry = new FlagEntry()
37            newEntry.FlagDate = gw.api.util.DateUtil.currentDate()
38            newEntry.Reason = FlagEntryReason.TC_OVERDUE_LEGAL_REPORT
39            relatedContact.addToFlagEntries(newEntry)
40            numberOfContactsWithOverdueReports++
41          }
```

Notes

Reporting messages without responses

Code example

Lines 24-27

- Query to check for messages that have not received a reply within a timely fashion.

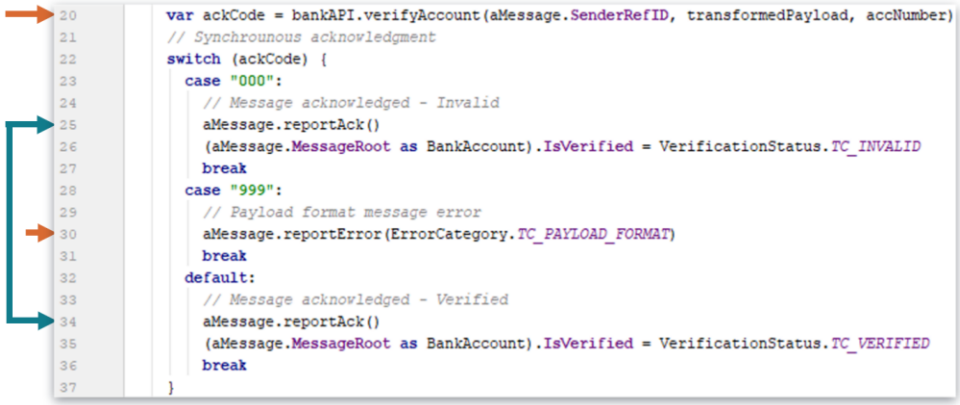
Line 38

- System flags the Contact with OVERDUE_LEGAL_REPORT

Periodically, the Message table should be checked for messages that have not received a reply within a timely fashion. This could be a Guidewire custom batch process or a database tool that monitors the Message table. Each message typically requires manual evaluation to determine the cause of the non-response and the appropriate steps to fix the situation.

Configure message acknowledgment mechanisms

Synchronous acknowledgement – Transport plugin



```
20  var ackCode = bankAPI.verifyAccount(aMessage.SenderRefID, transformedPayload, accNumber)
21  // Synchrounous acknowledgment
22  switch (ackCode) {
23    case "000":
24      // Message acknowledged - Invalid
25      aMessage.reportAck()
26      (aMessage.MessageRoot as BankAccount).IsVerified = VerificationStatus.TC_INVALID
27      break
28    case "999":
29      // Payload format message error
30      aMessage.reportError(ErrorCategory.TC_PAYLOAD_FORMAT)
31      break
32    default:
33      // Message acknowledged - Verified
34      aMessage.reportAck()
35      (aMessage.MessageRoot as BankAccount).IsVerified = VerificationStatus.TC_VERIFIED
36      break
37  }
```

The code can be found in the BankAccountVerificationTransport plugin **send** method

Notes

Synchronous acknowledgement – Transport plugin

Code example

Line 20
- Synchronous acknowledgment received returning acknowledgment code.

Lines 25,34
- Message is acknowledged based on ackCode.

Line 30
- Payload messaging error is reported.

When there is a bank account verification request from an external system, the message is acknowledged synchronously within the message transport plugin's send method. If there no verification (null value), you would want to capture and throw an exception and then report the error.

Asynchronous acknowledgement - Web service API

- External system replies using a published web service API
  - MessagingToolsAPI , or
  - CustomAPI
- Provides a message acknowledgement
  - Positive acknowledgement
  - Report external error
  - Duplicate

MessagingToolsAPI.gs

```
/**
 * Acknowledges message
 *
 * @param ack The acknowledgement to report
 * @throws SOAPException If the ack could not be committed to the database
 * @return true if the message was found and acked, false otherwise.
 */
@Throws(WsiAuthenticationException, "if there are permission/authentication issues")
@Throws(IllegalArgumentException, "if ack is invalid")
@Throws(SOAPSenderException, "If caller has not supplied valid acknowledgement")
@Throws(SOAPException, "If processing was in error")
function ackMessage(ack : Acknowledgement) : boolean {
  if (ack == null || ack.MessageID <= 0) {
    throw new IllegalArgumentException("Acknowledgement parameter is required")
  }
  return getDelegate().ackMessage( ack )
}
```

Notes

Asynchronous acknowledgement - Web service API

In the base application, **MessagingToolsAPI** can be used to acknowledge a pending message. However, for a given message, the only thing you can do is acknowledge it or report an error. You cannot execute any associated data changes. Custom web services provide the flexibility to execute any required logic, Guidewire recommends that such acknowledgements should be received through a custom web service API.

Asynchronous acknowledgement – Reply plugin (1)

- initTools method
  - Executed during reply plugin instance startup, helpful in processing asynchronous replies
- MessageFinder
  - Finds the message to be acknowledged based on unique identifier
    - Typically SenderRefID and DestinationID
- PluginCallbackHandler
  - Provides transactional context for:
    - Acknowledging the message
    - Any additional entity data changes

```
package trainingapp.demo.messaging

uses ...

class ExampleJMSReply implements MessageReply, MessageListener {
  var _finder: MessageFinder
  var _handler: PluginCallbackHandler
  var _destinationID: int
  var _senderRefID: String
  var _queueReceiver: QueueReceiver = null
  var _queueReceiverName = "entitychangereplies"
  static final var _MESSAGE_OK = "REQUEST_MESSAGE_OK"
  static final var _MESSAGE_DUPLICATE = "REQUEST_MESSAGE_DUPLICATE"
  static final var _MESSAGE_INVALID = "REQUEST_MESSAGE_INVALID"
  static final var _TEMP_UNAVAILABLE = "TEMP_UNAVAILABLE"
  override function initTools(pluginCallBackHandler: PluginCallbackHandler, messageFinder: MessageFinder) {
    _handler = pluginCallBackHandler
    _finder = messageFinder
  }
```

Notes

Asynchronous acknowledgement – Reply plugin (1)

initTools method specified in the reply plugin is executed during server startup. It provides resources that may be useful in processing replies. Like both the request and transport plugins, a reply plugin also includes the same set of lifecycle methods including resume, shutdown, and suspend. Reply plugins also have a set DestinationID property.

If multiple replies are received for the same message, the reply plugin implementation should not change system state or cause harm.

Asynchronous acknowledgement – Reply plugin (2)



```
42  override function onMessage(jmsMessage: javax.jms.Message) {
43    if (jmsMessage typeis TextMessage){
44      try {
45        _handler.execute(\-> {
46          var aMessage = _finder.findBySenderRefID(_senderRefID, _destinationID)
47          // handle duplicate
48          if (aMessage == null || jmsMessage.getStringProperty(_MESSAGE_DUPLICATE).HasContent || aMessage.AckCount > 0) {
49            var aMessageHistory = _finder.findHistoryBySenderRefID(_senderRefID, _destinationID)
50            if (aMessageHistory != null){
51              var currentTransaction = Transaction.getCurrent()
52              aMessageHistory = currentTransaction.add(aMessageHistory)
53              aMessageHistory.reportDuplicate()
54            }
55          }
```

Notes

Asynchronous acknowledgement – Reply plugin (2)

Code example

Line 42
- onMessage method executes message acknowledgment.

Line 45
- _handler provides transactional context.

Line 46
- Find pending acknowledgment message using SenderRefID and destination.

Lines 48-55
- Handle duplicate acknowledgment.

Asynchronous acknowledgement – Reply plugin (3)

Reply plugin – part three



```
56          // handle errors and acknowledgement
57          if (aMessage != null && aMessage.AckCount == 0) {
58            // code to handle message acknowledgement
59            if (jmsMessage.getStringProperty(_MESSAGE_INVALID).HasContent)  {
60              aMessage.reportError()
61            }
62            if (jmsMessage.getStringProperty(_TEMP_UNAVAILABLE).HasContent)  {
63              aMessage.reportError(typekey.ErrorCategory.TC_NO_CONNECTION)
64            }
65            if (jmsMessage.getStringProperty(_MESSAGE_OK).HasContent) {
66              aMessage.reportAck()
67            }
68          }
69        })
70      } catch (throwable : Exception) {
71        throwable.printStackTrace()// log in the realworld
72      }
73      jmsMessage.acknowledge()
74    }
```

Notes

Asynchronous acknowledgement – Reply plugin (3)

Code example

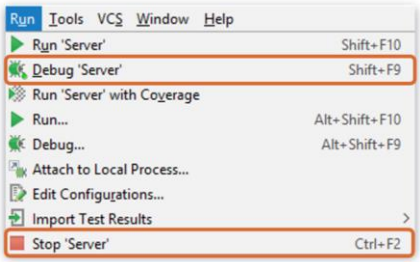
Lines 57-68
- Handle errors and acknowledgement based on error type.

Line 73
- Acknowledge the message. The message will be moved from the Message table to the History table.

Deploy code changes

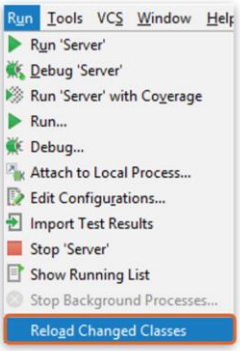
Restart Server

- New reply plugin class
- New or modified plugin registry
- New or modified destination



Reload Changed Classes

- Modified reply plugin class



Notes

Deploy code changes

You can restart the server from Studio by stopping a running server and after the server stops, then starting the server again in either Run or Debug mode (preferred for training).

What is a messaging acknowledgement from an external system?

A messaging acknowledgement is Guidewire's process for interpreting a messaging response from an external system.

What are the possible ways Guidewire can interpret an external system's response to a message?

Guidewire can interpret an external system's response, if one exists, as a positive acknowledgement (ACK), negative acknowledgment (NACK), or duplicate.

Under what three circumstances is an entry added to the MessageHistory table?

A message is added to the message history table when a message is positively acknowledged, skipped, or retried. For the first two, the message is removed entirely from the Message table. For the third, a copy of the original message remains in the Message table.

Under what circumstances are acknowledgements processed:

A) By the transport plugin?

B) By the reply plugin?

C) Without involvement from any plugin?

A) When the reply is received synchronously.

B) When the reply is received asynchronously through a listener or polling mechanism.

C) When the reply is received asynchronously by an external system call to an API.

When should you use each of the following methods?

A) aMessage.reportError(retryTime)?

B) aMessage.reportError(errorCategory)?

A) When the message needs to be retried and the message has not reached its maximum number of retries.

B) When the message needs to be retried, but the message has reached its maximum number of retries.